



RTL Design Sherpa

The Monitor System as a Design Surface AMBA Monitor System White Paper — Rev 1.1

October 2026

Table of Contents

1 The Monitor System as a Design Surface.....	5
2 The spine.....	6
3 Which monitor: the lite, unless you can say why not.....	7
4 Identity space allocation.....	9
5 Where to insert monitoring.....	11
6 Timestamp policy.....	13
7 Drain path selection.....	14
8 Packet-type filtering.....	16
9 Aggregation topology.....	17
10 What it costs.....	18
11 Validating a tweak in simulation.....	20
12 Out of scope.....	21
13 RTL AMBA Modules Index.....	22
13.1 Overview.....	22
13.2 AMBA 4 Protocol Implementations.....	22
13.2.1 WB4 (Wishbone B4, pipelined).....	22
13.2.2 APB4 (Advanced Peripheral Bus).....	23
13.2.3 AXI4-Full (Memory-Mapped Interface).....	23
13.2.4 AXI4-Lite (Register Interface).....	24
13.2.5 AXI4-Stream (Streaming Interface).....	25
13.3 AMBA 5 Protocol Implementations.....	25
13.3.1 APB5 (Advanced Peripheral Bus - AMBA 5).....	25

13.3.2 AXI5 (High-Performance Memory-Mapped - AMBA 5).....	26
13.3.3 AXI5-Stream (Streaming Interface - AMBA 5).....	27
13.4 Shared Infrastructure and Utilities.....	27
13.4.1 Generic AXI (GAXI) Components.....	27
13.4.2 Arbitration and Control.....	27
13.4.3 AXI Monitoring and Analysis.....	28
13.4.4 Monitor Bus Delivery + Bulk-Trace Compression.....	28
13.4.5 Memory / BRAM Slave (sdpram_slave family).....	29
13.4.6 Address and Data Processing.....	29
13.4.7 Clock and Reset Management.....	29
13.5 Protocol Bridge and Shims.....	29
13.5.1 AXI4 to APB Conversion.....	29
13.6 Monitor System.....	30
13.7 Package Definitions.....	30
13.7.1 Documented packages.....	30
13.7.2 The other two packages are documented, just not under their own names.....	30
13.8 Quick Reference.....	31
13.8.1 Module Count by Protocol.....	31
13.8.2 Performance Characteristics.....	31
13.8.3 AMBA 4 vs AMBA 5 Selection Guide.....	32
13.9 Usage Guidelines.....	32
13.9.1 Protocol Selection.....	32
13.9.2 Clock Gating Benefits.....	32

13.9.3 Design Integration.....	32
13.10 Navigation.....	32

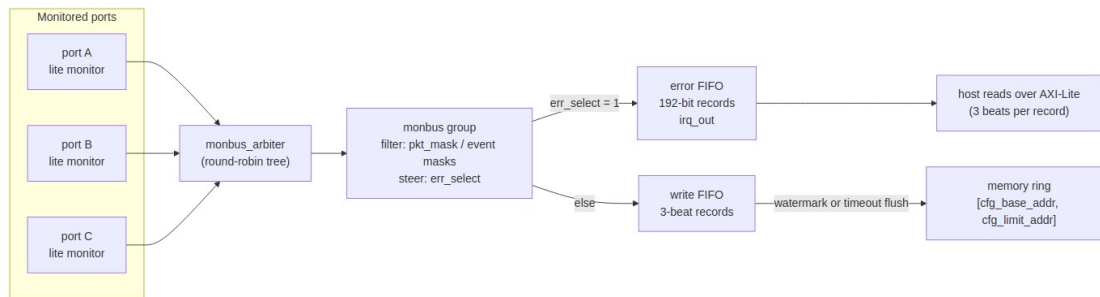
1 The Monitor System as a Design Surface

Audience: SoC integrators deciding how to instrument a design with the RTL Design Sherpa monitor system. **Not** a status snapshot of what is in the tree (that is [overview.md](#)) and not a port list (those are the per-module pages under [monitor/](#)). This paper sits one level up: here is the spine, here are the six axes the integrator owns, and here is what each choice costs.

Version: 1.1, 2026-09-28. Numbers are from the tree at that date; each is cited to the page or report it comes from. Revision 1.1 makes the recommendation explicit: the lite monitor is the realistic choice on every port, and the full monitor is the exception you justify.

2 The spine

Every monitored port emits 128-bit packets onto a monitor bus (monbus). An arbiter tree merges the ports of a block into one stream, and a monbus group sinks that stream into two paths: an error FIFO the host reads over AXI-Lite behind an interrupt, and a write FIFO the group flushes as AXI bursts into a memory ring. Filtering happens at the group, per protocol and per packet type. The packet layout is fixed ([monitor_package_spec.md](#)); everything else on this page is a knob.



Monitor system spine: ports, arbiter tree, group, two drains

3 Which monitor: the lite, unless you can say why not

Two monitors implement the port end of the spine. `axi_monitor_base`, the full monitor, tracks transactions in a CAM and drives six reporter cones (error, timeout, completion, threshold, performance, debug) with ID and address filtering. `axi_monitor_lite` tracks the same transactions in per-ID linked lists, emits errors, timeouts, completions with latency, thresholds and address-range events, and drops the performance and debug classes and the ID filter. Measured at their default parameters, out of context, on the Kintex-7 the boards ship on ([monitor/monitor_characterization.md](#)):

	Full monitor	Lite	Ratio
AXI4 read monitor, LUTs	7,006	1,115	6.3x
AXI4 read monitor, flops	5,219	1,027	5.1x
AXI-Lite read monitor, LUTs	2,891	929	3.1x
6.667 ns on Kintex-7 325T -2, register to register	misses (CAM at 16 slots)	meets, 0.8 to 1.5 ns spare	
10 ns on Artix-7 100T -1	misses by 1.45 ns	meets by 0.6 ns	

That is why the lite is the default: the bridge generator instantiates it on every monitored port, STREAM's three in-core monitors and RAPIDS' descriptor and AXIS monitors are lite, and every board build since 2026-09-26 carries it. It is also the more honest of the two under load. A full monitor that runs out of table entries stalls the command channel (`block_ready`); the lite never stalls the bus, counts the command it could not track (`refused_count`), and when its output queue is full it counts the events it could not deliver and reports the count in one `Error/EVENT_DROPPED` packet as soon as the bus is free. Its contract is exact and is proved in the tree: every command is completed, refused or live, and every generated event is delivered, reported dropped or pending, with no fourth outcome (`val/amba/test_axi_mon_block_ready.py`, `test_axi_monitor_soak.py`, `formal/amba/axi_monitor_lite`). A stalled bus is the one effect an instrument must never have on the design it watches.

What you give up, and when it matters:

Full-monitor feature	Lite	Reach for the full monitor if
performance windows (cfg_perf_enable, byte/beat/idle counters)	none; the in-datapath axi_bus_meter and the pass-through observer measure throughput without a monitor	you need windowed bandwidth numbers from the monitor itself
debug trace packets	none	you need per-beat trace, not events
ID-range filter (ID_FILTER_ENABLE)	none; one lite per port, sliced by unit_id/agent_id instead	several instances must share one port by ID
16 slots	8 by default, parameter	more than eight transactions in flight per port and every one must be tracked, not counted as refused
address filter	address ranges kept (N_ADDR_RANGES, error or match per range)	

The rest of this paper assumes the lite at the ports. Every axis below is the same for both monitors; the numbers are the lite's.

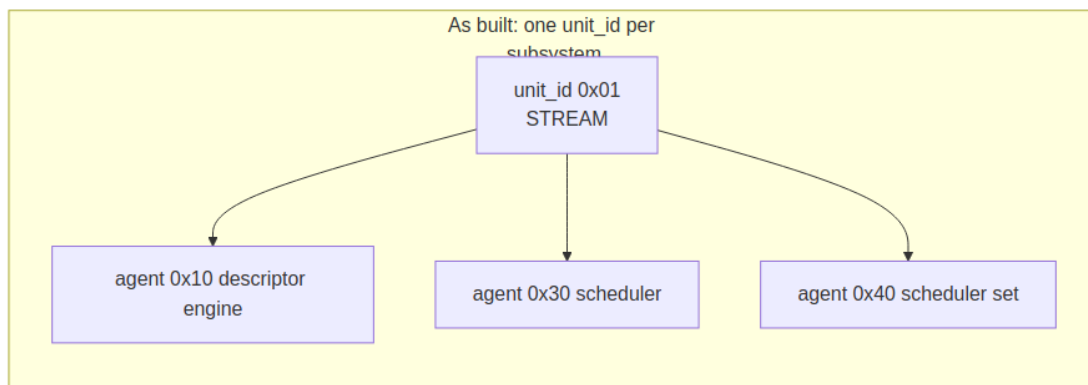
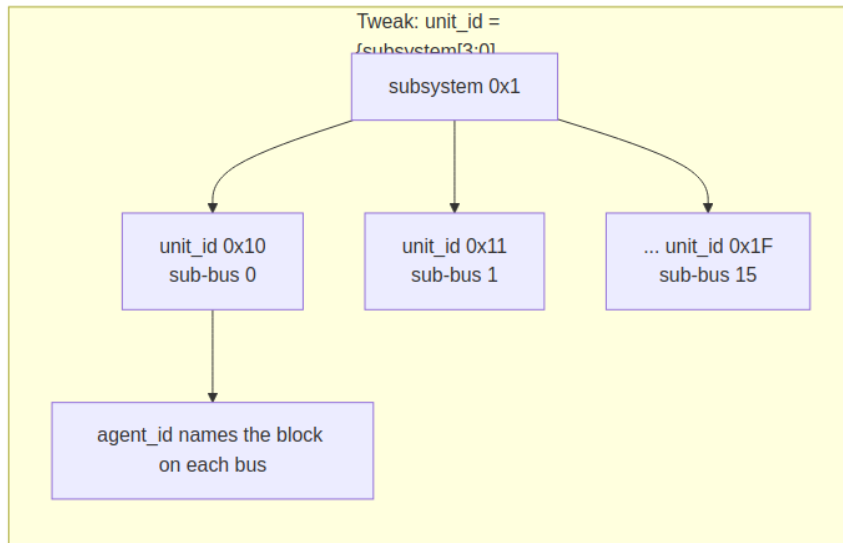
4 Identity space allocation

Three fields in every packet are designer-owned and fixed at build time:

Field	Bits	Set by	Meaning
unit_id	8	wrapper parameter UNIT_ID	which subsystem
agent_id	16	wrapper parameter AGENT_ID	which block inside it
channel_id	9	the monitor, from the transaction	which channel or AXI ID

The tools decode nothing from these fields; a host-side table maps the (unit_id, agent_id) tuple to a name. That makes the allocation yours. STREAM uses unit_id = 0x01 for the whole DMA and spends agent_id on blocks: 0x10 descriptor engine, 0x30 scheduler, 0x40 the widened scheduler set (projects/components/dma-ip/stream/rtl/macro/scheduler_group.sv).

Worked example, one level down. A subsystem with several internal buses can split unit_id into a subsystem nibble and a sub-bus nibble: unit_id = {sub_system[3:0], sub_bus[3:0]} gives sixteen subsystems each with up to sixteen tracked internal buses, and agent_id is still free to name the block on each bus. A host filter on unit_id[7:4] then isolates a subsystem, and on the full byte a single bus, with no change to the RTL. The cost is nothing: the fields exist in every packet regardless.

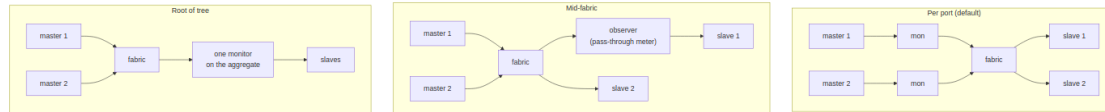


Identity allocation: one unit per subsystem, or a subsystem nibble and a sub-bus nibble

5 Where to insert monitoring

Placement	What it sees	Cost	Use when
Per port (the default; the bridge generator puts a lite monitor on every monitored port)	every transaction on that port, attributed by ID, with completion latency	one lite per port: about 1,100 LUTs / 1,030 FFs at defaults, 677 / 831 with the bridge's narrower parameters (monitor_characterization.md , axi_monitor_lite.md)	you need to know WHICH port misbehaved
Mid-fabric	the traffic crossing one internal link, through a pass-through observer with its own APB configuration (projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv)	one observer per link	localizing violations the fabric itself introduces (ordering, ID collisions between ports)
Root of tree	the aggregate of everything below, once	one monitor	area-constrained; you only need “something is wrong”, not where

Per port and root of tree are the ends of one trade: resolution against area. A three-port read bridge with per-port lite monitors spends about 2,000 LUTs on monitors and about 1,400 on the shared group; the same bridge monitored once at its root spends 677 and 1,400, and can no longer tell the ports apart. With the full monitor the same per-port choice would cost about 21,000 LUTs, which is why per-port monitoring was not realistic before the lite and is the default with it.



Insertion points: per port, mid-fabric, root of tree

6 Timestamp policy

A 64-bit timestamp rides beside every packet (`monbus_timestamp_t`). Today it is the monbus group family's local free-running counter (`mon_time_out`), distributed to the wrappers as `i_mon_time`, and the group writes its low 60 bits into the first beat of every three-beat record. Within one group every packet is on one clock; across groups there is no shared time base.

Two directions are open to the integrator:

- **Hybrid** `{global_us[47:0], local_cyc[15:0]}`: a system-wide microsecond count in the upper bits, the wrapper's own cycle count in the lower. Cross-subsystem correlation and per-wrapper cycle resolution then share the one field, and nothing in the packet layout moves.
- **External time source** (PTP or a chip-level counter): drive `i_mon_time` from it instead of the group's counter. The wrappers do not care where the value comes from.

Neither is prototyped; the hybrid form is the one this paper recommends when it is, because it needs no new field and no host-side re-basing.



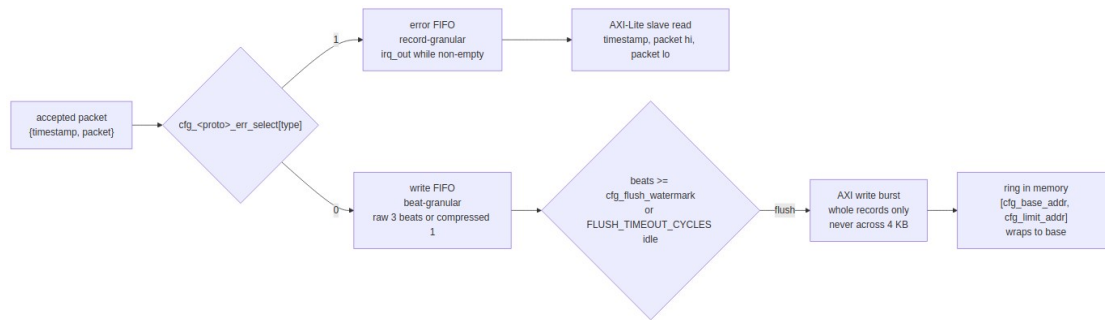
Timestamp policy: the group's local counter today, the hybrid time base as the tweak

7 Drain path selection

The group has two sinks and a per-type steering mask ([monbus_group.md](#)):

Sink	Record	Reached by	Best for
Error FIFO	192-bit {timestamp, packet}, read back as three 64-bit beats over the AXI-Lite slave; irq_out while non-empty	cfg_<proto>_err_select[type] = 1	the few packets a handler must act on now: errors, timeouts
Write FIFO	the same record as three beats (raw) or one beat (compressed), flushed as AXI bursts into the ring [cfg_base_addr, cfg_limit_addr] when cfg_flush_watermark beats are queued or the flush timeout elapses	everything not dropped and not selected for the error FIFO	bulk trace: completions, latencies, thresholds, for offline analysis

The choice is per packet type per protocol, at runtime. The common shape is errors and timeouts to the interrupt path and everything else to the trace ring; a debug session flips completions over to the error FIFO for a while and reads them one at a time.



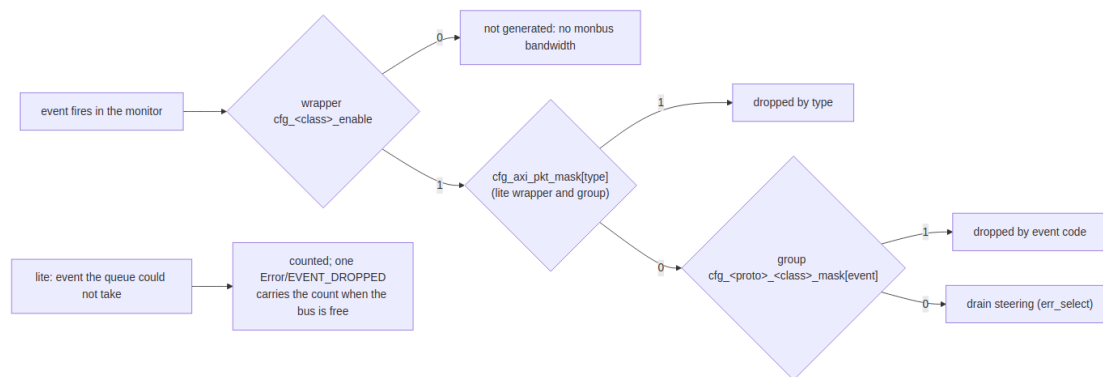
Drain paths: error FIFO with interrupt, write FIFO into the memory ring

8 Packet-type filtering

Three layers, all runtime-programmable over the control APB:

1. **Type mask** at the group, `cfg_<proto>_pkt_mask[type]`: 1 drops the type. Also on every lite wrapper (`cfg_axi_pkt_mask`) so a dropped type never even leaves the port.
2. **Event mask** at the group, `cfg_<proto>_<class>_mask[event_code[3:0]]`: 1 drops one event within a type, for example `RESP_SLVERR` but not `RESP_DECERR`.
3. **Enable pins** on the wrappers, `cfg_error_enable`, `cfg_compl_enable`, `cfg_timeout_enable`, `cfg_threshold_enable`: a class that is off is not generated at all, so it costs no monbus bandwidth.

The congestion pitfall, and how the lite retires it. On the full monitor, completion and performance packets are the high-rate classes; with both enabled on a busy port the monbus saturates and lower-priority packets are lost silently. The rule from [AXI_Monitor_Configuration_Guide.md](#) stands there: never enable `cfg_compl_enable` and `cfg_perf_enable` together. The lite has no performance class to collide with, holds a timeout or a latency event that loses the pick until it can go, and when its queue is genuinely full it counts the loss and reports the count in one `Error/EVENT_DROPPED` packet as soon as the bus is free. Nothing is lost silently; the consumer always knows how many events it did not see, which is the property a filtering strategy can be built on.

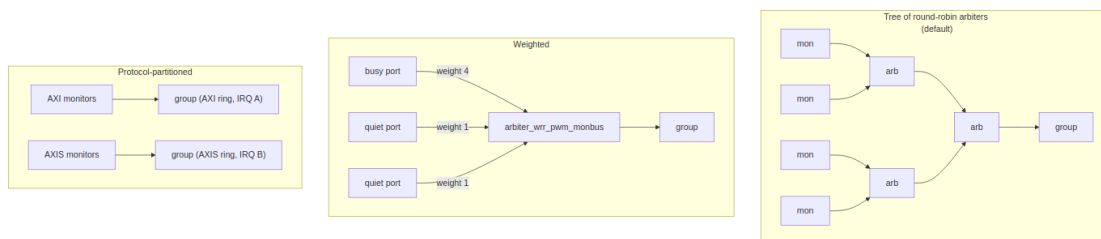


Packet-type filtering: enable pins, type mask, event mask, then drain steering; the lite counts what its queue could not take

9 Aggregation topology

Topology	Block	When
Tree of round-robin arbiters (default)	monbus_arbiter, any fan-in, ACK-mode grants	ports of similar rate; simplest, fair
Weighted	arbiter_wrr_pwm_monbus (weights per client, PWM flow control); arbiter_rr_pwm_monbus is the unweighted sibling	one port far busier than the others and you want its packets to win proportionally, not equally
Protocol-partitioned groups	one monbus_group_core per protocol family instead of the shared AXI / AXIS / CORE slots	the families need different rings, different interrupt handlers, or different flush policies

The group core is shared by every wrapper family (AXI4, AXI5, AXI-Lite, AXIS, Wishbone), so partitioning is a topology decision, not a new block.



Aggregation topologies: round-robin tree, weighted, protocol-partitioned

10 What it costs

Standalone, at default parameters, from the characterization sweep ([monitor/monitor_characterization.md](#)), and in place, from the bridge fixture with the bridge's parameters ([axi_monitor_lite.md](#)):

	Full monitor	Lite
One AXI4 read monitor at defaults (16 / 8 slots), LUTs / FFs	7,006 / 5,219	1,115 / 1,027
One read monitor in the bridge, LUTs / FFs	3,249 / 1,628	677 / 831
Three-port read bridge with group, LUTs	12,625	5,339
The unmonitored bridge, LUTs	826	826
Kintex-7 325T -2 at 6.667 ns, bridge WNS	+0.212 ns	+1.092 ns
Kintex-7 at 6.667 ns, standalone at defaults, register to register	-0.166 ns	+1.005 ns
A group (error FIFO 64 records, write FIFO 96 beats), LUTs / FFs	1,700 to 2,000 / 1,150 to 1,390	same, shared

The lite bridge fixture meets 10 ns on the Artix-7 100T -1 since 2026-09-28 (+0.366 ns), after the group planner and the lite's event stage were re-pipelined (amba ISSUE-001, monitor-lite ISSUE-002). The per-variant characterization – every wrapper family, with and without clock gating, full against lite, on both parts, from one repeatable flow – is [monitor/monitor_characterization.md](#). Its headline: at default parameters the full AXI

monitor is about 7,000 LUTs and the lite about 1,100, and the lite meets 150 MHz on the Kintex-7 with 0.8 ns or more to spare where the 16-slot full monitor does not.

11 Validating a tweak in simulation

Three templates in the tree exercise exactly the knobs above:

- `val/amba/test_axi_mon_block_ready.py`: saturate a port and prove the monitor's admission contract, `block_ready` on the full monitor and the refuse-count identity on the lite.
- `val/amba/test_axi_monitor_soak.py::monitor_soak_monlite`: random traffic with backpressure, asserting that every generated event is delivered or counted as dropped. Change a mask or a drain and this is the identity that must still close.
- `formal/amba/monbus_group_core`: the routing rule, `monbus_ready`, the FIFO accounting and the legality of the flush burst, proved at the ports. A change to the group's filtering or drain logic re-runs against it.

The error-injection template the original outline named (`test_bridge_1x2_rd_monitor_error_inject.py`) is not in the tree; the bridge's generated monitor stress tests cover that ground.

12 Out of scope

Packet bit layout ([monitor_package_spec.md](#)), per-module ports and timing ([monitor/](#)), and test recipes (the test sources).

[Back to Main Index](#) | [rtl-amba Overview](#)

13 RTL AMBA Modules Index

This directory contains documentation for the AMBA (Advanced Microcontroller Bus Architecture) RTL library, providing comprehensive implementations of ARM AMBA bus protocols. The library supports both **AMBA 4** and **AMBA 5** specifications across APB, AXI, AXI-Lite, and AXI-Stream protocols.

13.1 Overview

- **Overview** - Complete overview of the RTL AMBA library architecture and protocol implementations
- **Monitor system white paper** - the monitor system as a design surface: identity allocation, insertion points, timestamp policy, drain paths, filtering, aggregation topology, and what each costs. One chapter per file under `rtl-amba-monitors/whitepaper/`; branded PDF `docs/pdfs/RTL_AMBA_Monitor_Whitepaper.pdf` via `generate_rtl_pdfs.sh monitor-wp`.

13.2 AMBA 4 Protocol Implementations

AMBA 4 modules provide the foundation for high-performance SoC interconnect, widely deployed in production systems. These modules are mature, fully verified, and optimized for synthesis.

13.2.1 WB4 (Wishbone B4, pipelined)

Open-standard peripheral bus; the pair speaks B4 pipelined mode behind the same command/response queues as the APB4 pair. There is no Wishbone B5.

Wishbone B4 Module Documentation

- **wb4_master** - Pipelined Wishbone master with command/response queues; credit-gated issue, RTY reported as status
- **wb4_slave** - Pipelined Wishbone slave with command/response queues; in-order registered termination, orphan-response guard
- **wb4_monitor** - Monitor on the wb4 queues emitting PROTOCOL_WB monitor bus packets; in-order tracking, timeouts, latency, address-range check
- **wb4_retry** - RTY retry block on the FUB side of the master; in-order completion buffer, budget and delay
- **wb4_master_retry** - `wb4_retry` + `wb4_master` with the master's ports

- [wb4_master_cg](#) / [wb4_slave_cg](#) - Clock-gated variants (amba_clock_gate_ctrl, runtime enable/idle count)
- [wb4_slave_cdc](#) - Slave with its queues crossed to another clock domain (two gaxi_fifo_async)

13.2.2 APB4 (Advanced Peripheral Bus)

Simple, low-power peripheral bus for control registers and low-bandwidth devices.

APB4 Module Documentation

13.2.2.1 Core Components

- [apb4_master](#) - APB master with command/response interfaces and FIFO buffering
- [apb4_master_cg](#) - Clock-gated APB master for power optimization
- [apb4_slave](#) - APB slave with configurable address decoding and response generation
- [apb4_slave_cg](#) - Clock-gated APB slave for power optimization
- [apb4_monitor](#) - APB protocol monitor for verification and debugging

13.2.2.2 Clock Domain Crossing

- [apb4_slave_cdc](#) - APB slave with clock domain crossing support
- [apb4_slave_cdc_cg](#) - Clock-gated CDC APB slave

13.2.2.3 Test and Development

- [apb4_master_stub](#) - APB master stub for testing and simulation
 - [apb4_slave_stub](#) - APB slave stub for testing and simulation
-

13.2.3 AXI4-Full (Memory-Mapped Interface)

High-performance memory-mapped interface supporting burst transactions and multiple outstanding operations.

AXI4 Module Documentation

13.2.3.1 Master Components

- [axi4_master_rd](#) - AXI4 read master with skid buffers and burst support
- [axi4_master_rd_cg](#) - Clock-gated AXI4 read master
- [axi4_master_wr](#) - AXI4 write master with address/data coordination

- [axi4_master_wr_cg](#) - Clock-gated AXI4 write master

13.2.3.2 *Slave Components*

- [axi4_slave_rd](#) - AXI4 read slave with configurable response handling
- [axi4_slave_rd_cg](#) - Clock-gated AXI4 read slave
- [axi4_slave_wr](#) - AXI4 write slave with write response generation
- [axi4_slave_wr_cg](#) - Clock-gated AXI4 write slave

13.2.3.3 *Test Stubs*

- [axi4_master_stub](#) - Complete AXI4 master stub
 - [axi4_master_rd_stub](#) - AXI4 read master stub
 - [axi4_master_wr_stub](#) - AXI4 write master stub
 - [axi4_slave_stub](#) - Complete AXI4 slave stub
 - [axi4_slave_rd_stub](#) - AXI4 read slave stub
 - [axi4_slave_wr_stub](#) - AXI4 write slave stub
-

13.2.4 AXI4-Lite (Register Interface)

Simplified AXI4 for register access with single outstanding transaction support.

AXI4-Lite Module Documentation

13.2.4.1 *Master Components*

- [axil4_master_rd](#) - AXI4-Lite read master for register access
- [axil4_master_rd_cg](#) - Clock-gated AXI4-Lite read master
- [axil4_master_wr](#) - AXI4-Lite write master for register access
- [axil4_master_wr_cg](#) - Clock-gated AXI4-Lite write master

13.2.4.2 *Slave Components*

- [axil4_slave_rd](#) - AXI4-Lite read slave with register interface
 - [axil4_slave_rd_cg](#) - Clock-gated AXI4-Lite read slave
 - [axil4_slave_wr](#) - AXI4-Lite write slave with register interface
 - [axil4_slave_wr_cg](#) - Clock-gated AXI4-Lite write slave
-

13.2.5 AXI4-Stream (Streaming Interface)

High-throughput unidirectional streaming for data processing pipelines.

AXI4-Stream Module Documentation

13.2.5.1 Stream Components

- [axis4_master](#) - AXI4-Stream master for data streaming
 - [axis4_master_cg](#) - Clock-gated AXI4-Stream master
 - [axis4_slave](#) - AXI4-Stream slave for data reception
 - [axis4_slave_cg](#) - Clock-gated AXI4-Stream slave
-

13.3 AMBA 5 Protocol Implementations

AMBA 5 modules provide enhanced features for next-generation SoC designs, including atomic operations, extended QoS, memory tagging, and improved power management signaling.

13.3.1 APB5 (Advanced Peripheral Bus - AMBA 5)

Enhanced APB with wake-up signaling, non-secure extension, and user signals.

APB5 Module Documentation

Feature	APB4	APB5
Protection	PPROT[2:0]	PPROT[2:0] + PNSE
Wake-up	Not supported	PWAKEUP signal
User Signals	Not supported	PAUSER, PWUSER, PRUSER, PBUSER

13.3.1.1 Core Components

- [apb5_master](#) - APB5 master with command/response interface
- [apb5_slave](#) - APB5 slave with buffered cmd/rsp interface
- [apb5_slave_cdc](#) - APB5 slave with clock domain crossing
- [apb5_monitor](#) - APB5 transaction monitor

13.3.1.2 Clock-Gated Variants

- [apb5_master_cg](#) - Clock-gated APB5 master
- [apb5_slave_cg](#) - Clock-gated APB5 slave
- [apb5_slave_cdc_cg](#) - Clock-gated CDC APB5 slave

13.3.1.3 Test Stubs

- [apb5_master_stub](#) - APB5 master stub for testing

- [apb5_slave_stub](#) - APB5 slave stub for testing
-

13.3.2 AXI5 (High-Performance Memory-Mapped - AMBA 5)

Enhanced AXI with atomic operations, memory tagging, and extended QoS.

AXI5 Module Documentation

Feature	AXI4	AXI5
Outstanding	Up to 16	Up to 256
Atomic Ops	Not supported	AtomicStore, AtomicLoad, AtomicSwap, AtomicCompare
Memory Tagging	Not supported	AWMEMATTR, ARMEMATTR
Data Poisoning	Not supported	RPOISON, WPOISON

13.3.2.1 Master Components

- [axi5_master_rd](#) - AXI5 read master with skid buffers
- [axi5_master_rd_cg](#) - Clock-gated AXI5 read master
- [axi5_master_wr](#) - AXI5 write master with address/data coordination
- [axi5_master_wr_cg](#) - Clock-gated AXI5 write master

13.3.2.2 Slave Components

- [axi5_slave_rd](#) - AXI5 read slave with configurable response
- [axi5_slave_rd_cg](#) - Clock-gated AXI5 read slave
- [axi5_slave_wr](#) - AXI5 write slave with response generation
- [axi5_slave_wr_cg](#) - Clock-gated AXI5 write slave

13.3.2.3 Integrated Monitor Modules

- [axi5_master_rd_mon](#) - Read master with integrated monitor
- [axi5_master_rd_mon_cg](#) - Clock-gated read master with monitor
- [axi5_master_wr_mon](#) - Write master with integrated monitor
- [axi5_master_wr_mon_cg](#) - Clock-gated write master with monitor
- [axi5_slave_rd_mon](#) - Read slave with integrated monitor
- [axi5_slave_rd_mon_cg](#) - Clock-gated read slave with monitor

- [axi5_slave_wr_mon](#) - Write slave with integrated monitor
 - [axi5_slave_wr_mon_cg](#) - Clock-gated write slave with monitor
-

13.3.3 AXI5-Stream (Streaming Interface - AMBA 5)

Enhanced streaming with wake-up signaling and data poisoning support.

AXI5-Stream Module Documentation

Feature	AXI4-Stream	AXI5-Stream
Wake-up	Not supported	TWAKEUP signal
Data Poisoning	Not supported	TPOISON

13.3.3.1 Stream Components

- [axis5_master](#) - AXI5-Stream master for data streaming
 - [axis5_master_cg](#) - Clock-gated AXI5-Stream master
 - [axis5_slave](#) - AXI5-Stream slave for data reception
 - [axis5_slave_cg](#) - Clock-gated AXI5-Stream slave
-

13.4 Shared Infrastructure and Utilities

Infrastructure components used across all AMBA protocols.

13.4.1 Generic AXI (GAXI) Components

GAXI Module Documentation

- [gaxi_fifo_sync](#) - Synchronous FIFO for GAXI interfaces
- [gaxi_fifo_async](#) - Asynchronous FIFO for clock domain crossing
- [gaxi_skid_buffer](#) - Skid buffer for pipeline optimization
- [gaxi_skid_buffer_async](#) - Asynchronous skid buffer

13.4.2 Arbitration and Control

- [arbiter_monbus_common](#) - Common arbitration logic for monitoring bus
- [arbiter_rr_pwm_monbus](#) - Round-robin PWM arbiter for monbus
- [arbiter_wrr_pwm_monbus](#) - Weighted round-robin PWM arbiter
- [monbus_arbiter](#) - Monitoring bus arbiter

13.4.3 AXI Monitoring and Analysis

- **axi_monitor_base** - Base AXI/AXIL transaction monitor (per-DMA)
- **axi_monitor_reporter** - Thin dispatcher + 6 packet-type sub-blocks (error / timeout / compl / threshold / perf / debug) gated by ENABLE_*_LOGIC
- **axi_monitor_timeout** - AXI timeout detection
- **axi_monitor_timer** - AXI timing analysis
- **axi_monitor_trans_mgr** - AXI transaction-table management (CAM-backed)
- **monitor_trans_cam** - Multi-port ID CAM with opaque payload (backs trans_mgr)
- **axi_monitor_addr_check** - Per-channel address-match watchpoints
- **axi_monitor_lite** - The AXI/AXIL transaction monitor at a fifth of the gates; behind every_monlite wrapper
- **axis_monitor_lite** - The AXI4-Stream monitor in the lite discipline: packets, stalls, bubbles, TID/TDEST changes as Stream/Credit/Channel/Error/Timeout/Completion packets
- **axi4_intf_master_observer** - Standalone, protocol-agnostic interface observer (wraps any AXI4 master interface from outside; companion to the per-DMA axi_monitor_* family). Includes axi_bus_meter and axi_perf_latency_hist per port. Lives at projects/components/utility-ip/misc/rtl/axi4_intf_master_observer.sv; the axi4_dma_observer copy in rtl/amba/shared/ was retired 2026-08-14.

13.4.4 Monitor Bus Delivery + Bulk-Trace Compression

Path from in-flight monitor packets to host-visible memory + CPU IRQs.

- **monbus_group (family)** - Protocol-agnostic core + four protocol-permutation wrappers:
 - monbus_axil4_axil4_group (AXIL slave-read + AXIL master-write, replaces legacy monbus_axil_group)
 - monbus_axil4_axi4_group (AXIL slave-read + AXI4 burst master-write)
 - monbus_axi4_axil4_group (AXI4 burst slave-read + AXIL master-write)
 - monbus_axi4_axi4_group (burst on both sides)

- [monbus_compressor](#) - Bulk-trace encoder, 2-stage pipeline, per-template delta_ts, bit-exact to Python golden (~2.6× ratio)
- [monbus_cam](#) - 32-entry LRU CAM with per-entry 24b last_ts (backs the compressor)
- [monbus_pkt_tally](#) - on-chip packet-type coverage histogram: SRAM count matrix keyed by {protocol, pkt_type, event_code}, 32-entry LRU write-combining cache in front (reuses monbus_cam), saturating counts, freeze/flush snapshot + first-event latch

13.4.5 Memory / BRAM Slave (sdpram_slave family)

- [sdpram_slave \(family\)](#) - Protocol-agnostic BRAM backend + four protocol-permutation wrappers (sdpram_slave_axi4_axi4, sdpram_slave_axi4_axil, sdpram_slave_axil_axi4, sdpram_slave_axil_axil). Canonical memory-dump-ring backend for the monbus group's master-write port.

13.4.6 Address and Data Processing

- [axi_gen_addr](#) - AXI address generation utility
- [axi_split_combi](#) - AXI combinational splitting logic
- [axi_master_rd_splitter](#) - AXI read master splitter
- [axi_master_wr_splitter](#) - AXI write master splitter

13.4.7 Clock and Reset Management

- [amba_clock_gate_ctrl](#) - AMBA-specific clock gating controller
 - [Clock Domain Crossing](#) - Single reference for all CDC: decision guide, reset behavior, and every building block (cdc_synchronizer, cdc_open_loop, cdc_2_phase_handshake, cdc_4_phase_handshake, async FIFO pointers)
-

13.5 Protocol Bridge and Shims

13.5.1 AXI4 to APB Conversion

- [axi4_to_apb4_convert](#) - AXI4 to APB protocol converter
 - [axi4_to_apb4_shim](#) - Lightweight AXI4 to APB shim
-

13.6 Monitor System

Start with [The Monitor System](#) – the architecture and what it can do: error/debug/perf packet production for protocols *and* for custom blocks like arbiters, source-side filtering, the two transports, and the three capture strategies (bulk trace, compressed trace, on-chip counting).

13.7 Package Definitions

13.7.1 Documented packages

- [monitor_amba4_pkg](#) - AMBA4 monitor types and packet layout
- [monitor_amba5_pkg](#) - AMBA5 monitor types and packet layout
- [monitor_arbiter_pkg](#) - Monitor-bus arbitration types
- [monitor_package_spec](#) - The package family, specified

13.7.2 The other two packages are documented, just not under their own names

`monitor_common_pkg` and `monitor_pkg` have no same-named page and do not need one – [monitor_package_spec](#) IS their page:

- **monitor_common_pkg** (165 importers, the workhorse) is the subject of that whole document – the 128-bit record, the protocol and packet-type enums, the type aliases, the helper functions, the side-band timestamp.
- **monitor_pkg** (30 importers) defines nothing of its own. It is a backward-compatibility shim that re-exports the four split packages, and it is covered by that page's *Backward Compatibility* section.

A second page per package would be a second copy of the same content, which is how documentation rots. Measured 2026-08-31; do not re-raise this as a gap on the strength of a filename search.

`apb4_pkg.sv`, `apb5_pkg.sv`, `axi_pkg.sv` and `bus_types.svh` were DELETED the same day – all four had zero importers (`apb4_pkg` was imported only by `apb5_pkg`, and `bus_types.svh`, which included two of them, was included by nothing). They were a vestigial type library, not undocumented code. (`monitor_network_pkg` never had RTL at all – it was always a phantom.)

13.8 Quick Reference

13.8.1 Module Count by Protocol

Protocol	AMBA 4	AMBA 5	Total
APB	9 modules	9 modules	18
AXI (Full)	14 modules	16 modules	30
AXI-Lite	8 modules	-	8
AXI-Stream	4 modules	4 modules	8
GAXI	5 modules	-	5
Infrastructure			
Shared Utilities	15 modules	-	15
Protocol Bridges	2 modules	-	2

Total: 86 AMBA modules

13.8.2 Performance Characteristics

Protocol	Typical Frequency	Throughput	Use Case
APB4/APB5	100-200 MHz	400 MB/s - 1.6 GB/s	Control registers, low-speed peripherals
AXI4-Lite	200-400 MHz	800 MB/s - 3.2 GB/s	Register access, configuration
AXI4/AXI5	300-500 MHz	9.6 GB/s - 32 GB/s	High-performance memory access
AXI4/5-Stream	400-600 MHz	12.8 GB/s - 76.8 GB/s	Data streaming, packet processing

13.8.3 AMBA 4 vs AMBA 5 Selection Guide

Requirement	Recommendation
Proven, mature design	AMBA 4
Atomic operations needed	AMBA 5 (AXI5)
Memory tagging (MTE)	AMBA 5 (AXI5)
Power management wake-up	AMBA 5 (APB5, AXI5-Stream)
TrustZone integration	AMBA 5 (APB5 PNSE)
Maximum compatibility	AMBA 4
Next-generation features	AMBA 5

13.9 Usage Guidelines

13.9.1 Protocol Selection

- **APB4/APB5:** Simple register access, low area/power requirements
- **AXI4-Lite:** Standard register interface, moderate performance
- **AXI4/AXI5:** High-performance memory access, burst transactions
- **AXI4/5-Stream:** High-throughput data streaming, packet processing

13.9.2 Clock Gating Benefits

- **Power Reduction:** 20-40% dynamic power savings
- **Conditional Operation:** Clock gating when interfaces are idle
- **Fine-Grained Control:** Per-channel clock gating available

13.9.3 Design Integration

```
// Example: Complete AXI5 system with monitoring
axi5_master_rd_mon #(.AXI_DATA_WIDTH(64)) u_rd_master (...);
axi5_master_wr_mon #(.AXI_DATA_WIDTH(64)) u_wr_master (...);
axi5_slave_rd      #(.AXI_DATA_WIDTH(64)) u_rd_slave (...);
axi5_slave_wr      #(.AXI_DATA_WIDTH(64)) u_wr_slave (...);
```

13.10 Navigation

- [Back to RTL Documentation](#) - Return to main RTL documentation index